Object-Oriented Programming Project Report

University System: users, courses, grading, research, news, authentication, and persistence.

Project Information

PROJECT **University Management System**	LANGUAGE **Java**
ARCHITECTURE **Model-Service-Repository**	MAIN PACKAGE ROOT **project**
PREPARED BY **Danil Goncharov(24B031726), Darkhan Izbasarov(24B031815)**	DATE **May 17, 2026**

Table of Contents

1. Abstract
2. Project Objectives
3. Architecture Overview
4. UML Diagrams
5. Class and Interface Descriptions
6. Code Fragments
7. OOP Principles and Design Patterns
8. Project Management Notes
9. Screenshots
10. Conclusion

1. Abstract

This report documents a Java-based university management system developed with object-oriented programming principles. The system models students, teachers, managers, administrators, courses, transcripts, marks, research activities, news notifications, employee requests, authentication, and serialized persistence.

The application uses a clear package structure: domain entities are stored in `model`, enumerations in `enums`, custom checked exceptions in `exceptions`, business services and comparators in `service`, and centralized persistence in `repository`. UML diagrams are included to show static relationships, runtime object examples, and use-case behavior.

2. Project Objectives

- Model the core roles of a university system: students, teachers, managers, administrators, and researchers.
- Implement course registration, teacher assignment, transcript storage, grade calculation, and GPA recalculation.
- Support staff communication, employee requests, news publishing, and observer-based notifications.
- Provide research-related operations such as papers, projects, h-index calculation, and citation-based ranking.

- Apply OOP concepts including inheritance, polymorphism, encapsulation, interfaces, abstraction, composition, and exceptions.
- Persist application state through a singleton database serialized to `database.ser`.

3. Architecture Overview

Model Layer

Entities such as `User`, `Student`, `Teacher`, `Course`, and `ResearchPaper`.

Service Layer

Authentication, research operations, sorting comparators, and user creation factory.

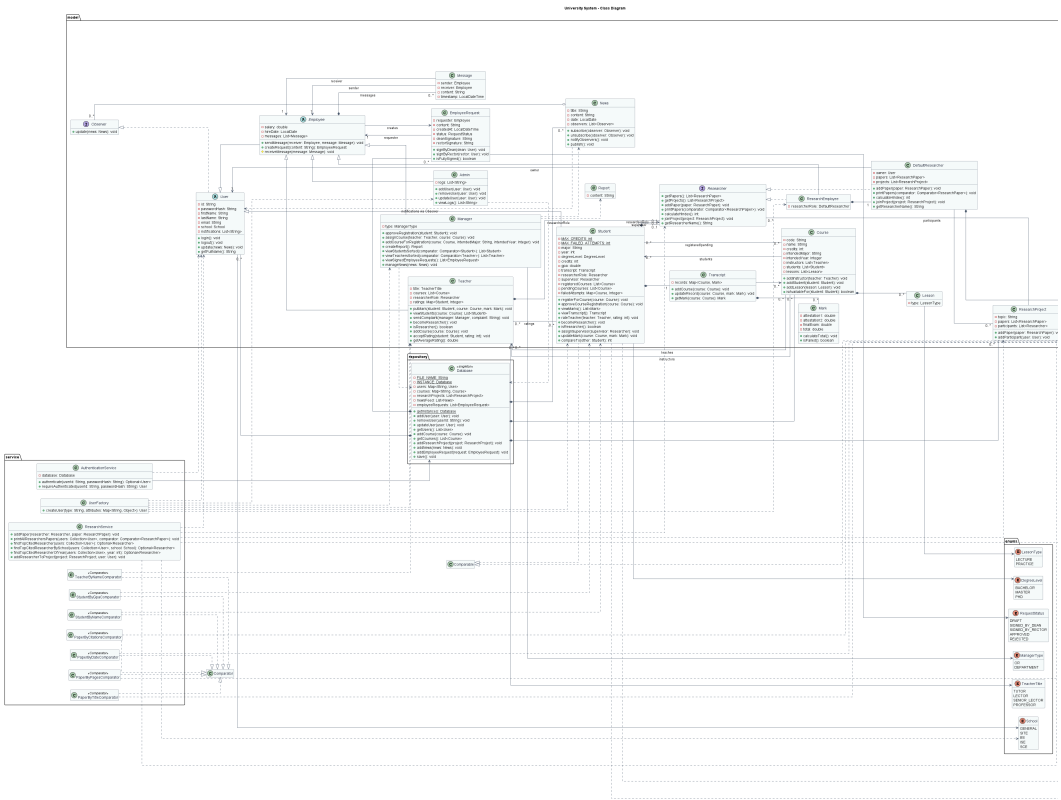
Repository Layer

Singleton `Database` storing users, courses, projects, news, and requests.

The domain model is built around an inheritance hierarchy rooted in `User`. Staff roles inherit from `Employee`, while `Student` extends `User` directly. Research behavior is extracted into a `Researcher` interface so it can be implemented by `ResearchEmployee` directly or delegated to `DefaultResearcher` for students and teachers.

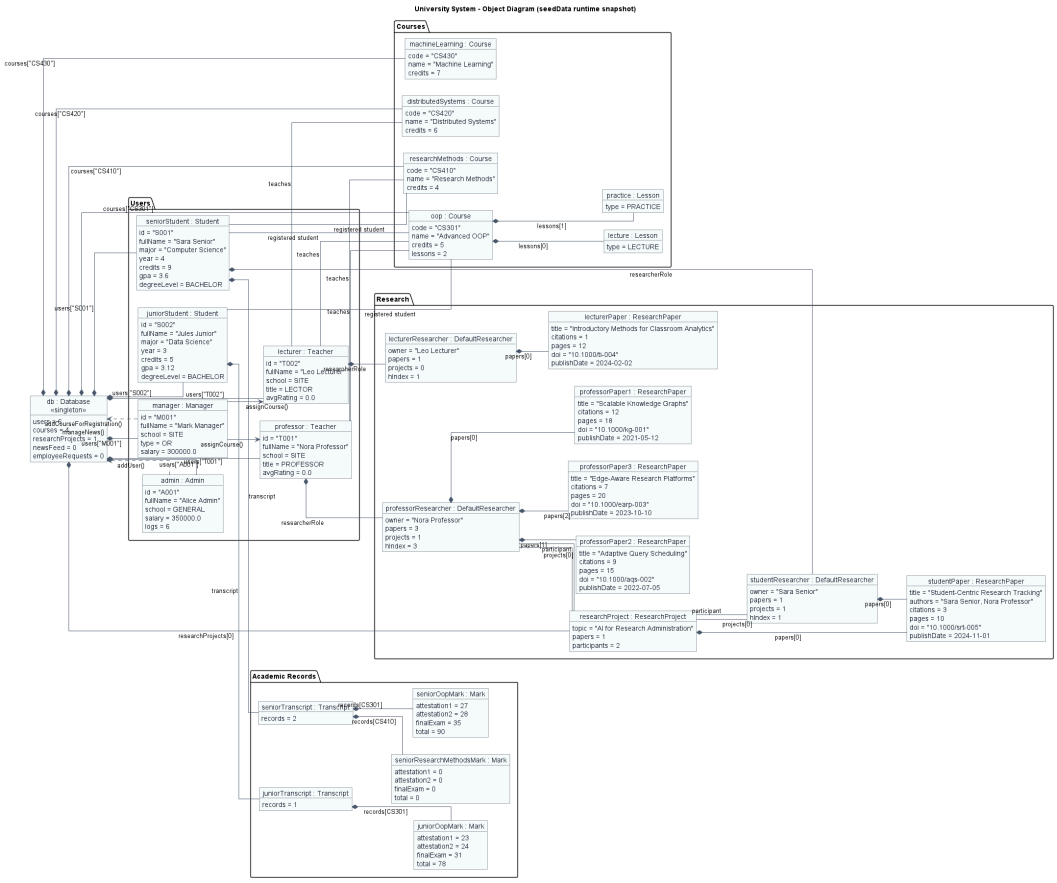
4. UML Diagrams

4.1 Class Diagram



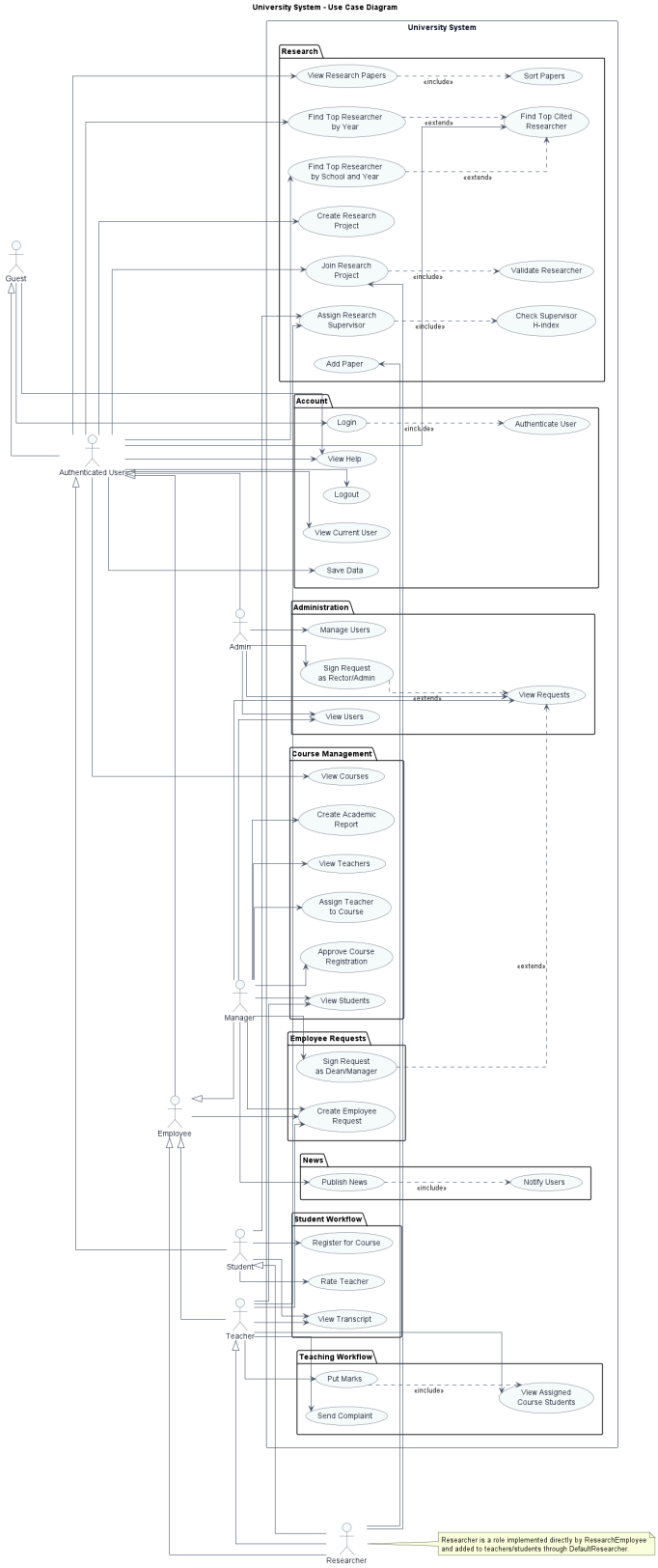
The class diagram shows packages, inheritance, interface implementation, composition, dependencies, repository relationships, comparators, enums, and custom exceptions.

4.2 Object Diagram



The object diagram illustrates a possible runtime state with concrete users, courses, messages, papers, projects, news, and database collections.

4.3 Use-Case Diagram



The use-case diagram summarizes how students, teachers, managers, administrators, researchers, and employees interact with the system.

5. Class and Interface Descriptions

5.1 Core User Hierarchy

User

abstract class model

Purpose: Base abstraction for all users in the system. It stores ID, password hash, name, email, school, and notifications.

Important behavior: Implements `Observer` so users can receive `News` updates. Provides `login()`, `logout()`, and `getFullName()`.

Employee

abstract class model

Purpose: Shared superclass for staff roles. Adds salary, hire date, direct messages, and request creation.

Important behavior: `sendMessage()` stores a message for sender and receiver, while `createRequest()` adds an `EmployeeRequest` to the database.

Student

class Comparable

Purpose: Represents a learner with major, year, degree level, credits, GPA, transcript, courses, and optional researcher role.

Important behavior: Validates course availability and credit limit during registration, tracks pending registrations, updates marks, recalculates GPA, rates teachers, and assigns research supervisors.

Teacher

class model

Purpose: Represents teaching staff responsible for courses, marks, complaints, ratings, and optional research activity.

Important behavior: Ensures the teacher is assigned to a course before grading students. Professors automatically become researchers.

Manager

class model

Purpose: Coordinates academic administration, including course registration, teacher assignment, reports, signed requests, and news publishing.

Important behavior: Uses `Database` to collect report data and publish news to all users through the observer mechanism.

Admin

class model

Purpose: Manages user accounts and records actions in a local admin log.

Important behavior: Delegates add, remove, and update operations to the singleton `Database`.

ResearchEmployee

class Researcher

Purpose: Employee role with direct research capabilities.

Important behavior: Delegates paper and project operations to an internal `DefaultResearcher`.

5.2 Academic and Communication Classes

Class / Interface	Type	Description	Key Methods / Fields
`Course`	Class	Academic course with code, name, credits, intended major/year, instructors, students, and lessons.	`addInstructor`, `addStudent`, `addLesson`, `isAvailableFor`
`Lesson`	Class	Represents a lesson belonging to a course.	`LessonType type`
`Transcript`	Class	Stores a map of courses to marks for a student.	`addCourse`, `updateRecord`, `getMark`
`Mark`	Class	Stores attestation and final exam grades, calculates total, and determines failure.	`calculateTotal`, `isFailed`
`Message`	Class	Represents staff communication between sender and receiver.	`sender`, `receiver`, `content`, `timestamp`
`EmployeeRequest`	Class	Formal employee request with dean and rector signature workflow.	`signByDean`, `signByRector`, `isFullySigned`
`News`	Class	Announcement entity that supports observer subscriptions and publishing.	`subscribe`, `unsubscribe`, `notifyObservers`, `publish`
`Report`	Class	Simple container for generated report text.	`content`
`Observer`	Interface	Notification interface implemented by users.	`update(News news)`

5.3 Research Classes and Interfaces

Class / Interface	Type	Description	Key Methods / Fields
`Researcher`	Interface	Defines the contract for any object that can participate in research.	`addPaper`, `calculateHIndex`, `joinProject`
`DefaultResearcher`	Class	Reusable implementation of research behavior owned	`owner`, `papers`, `projects`

Class / Interface	Type	Description	Key Methods / Fields
		by a user.	
`ResearchPaper`	Class	Publication with title, authors, journal, citations, pages, DOI, and publish date.	`Comparable<ResearchPaper>`, `compareTo`
`ResearchProject`	Class	Research project with topic, papers, and researcher participants.	`addPaper`, `addParticipant`

5.4 Repository, Services, Comparators, Enums, and Exceptions

Name	Category	Description
`Database`	Repository	Singleton persistent store for users, courses, research projects, news, and employee requests.
`AuthenticationService`	Service	Validates user ID and password hash against the database.
`ResearchService`	Service	Finds researchers, adds papers, ranks citation leaders, and adds researchers to projects.
`UserFactory`	Factory Service	Creates concrete user objects from a type string and attribute map.
`PaperByCitationsComparator`	Comparator	Sorts research papers by citation count.
`PaperByDateComparator`	Comparator	Sorts research papers by publication date.
`PaperByPagesComparator`	Comparator	Sorts research papers by page count.
`PaperByTitleComparator`	Comparator	Sorts research papers alphabetically by title.
`StudentByGpaComparator`	Comparator	Sorts students by GPA.
`StudentByNameComparator`	Comparator	Sorts students by full name.
`TeacherByNameComparator`	Comparator	Sorts teachers by full name.
`DegreeLevel`, `LessonType`, `ManagerType`, `RequestStatus`, `School`, `TeacherTitle`	Enums	Constrained values for degree, lesson, manager, request, school, and teacher title states.
`CreditLimitExceededException`	Exception	Thrown when a student exceeds the maximum credit limit.
`LowHIndexException`	Exception	Thrown when a supervisor does not meet the required h-index.

Name	Category	Description
`NotAResearcherException`	Exception	Thrown when a non-researcher is added to a research project.

6. Code Fragments

Course registration with credit and course availability checks

```
public void registerForCourse(Course course) throws CreditLimitExceededException {
    if (course == null || registeredCourses.contains(course) ||
pendingCourses.contains(course)) {
        return;
    }
    if (!course.isAvailableFor(this)) {
        throw new IllegalArgumentException("Course is not intended for this student's
major/year.");
    }
    int plannedCredits = credits + pendingCourses.stream()
            .mapToInt(Course::getCredits)
            .sum() + course.getCredits();
    if (plannedCredits > MAX_CREDITS) {
        throw new CreditLimitExceededException("Credit limit exceeded.");
    }
    pendingCourses.add(course);
}
```

Authentication service with Optional result

```
public Optional<User> authenticate(String userId, String passwordHash) {
    User user = database.getUser(userId);
    if (user == null || !user.getPasswordHash().equals(passwordHash)) {
        return Optional.empty();
    }
    user.login();
    return Optional.of(user);
}
```

6. Code Fragments Continued

Singleton database loading and saving

```
private static Database INSTANCE = loadDatabase();

public static Database getInstance() {
    return INSTANCE;
}

public synchronized void save() {
    try (ObjectOutputStream outputStream =
            new ObjectOutputStream(new FileOutputStream(FILE_NAME))) {
        outputStream.writeObject(this);
    } catch (IOException e) {
        throw new IllegalStateException("Unable to save database.", e);
    }
}
```

Researcher extraction across different user types

```
public Optional<Researcher> extractResearcher(User user) {
    if (user instanceof Researcher directResearcher) {
        return Optional.of(directResearcher);
    }
    if (user instanceof Student student && student.isResearcher()) {
        return Optional.of(student.getResearcherRole());
    }
    if (user instanceof Teacher teacher && teacher.isResearcher()) {
        return Optional.of(teacher.getResearcherRole());
    }
    return Optional.empty();
}
```

Observer notification through News

```
public void notifyObservers() {
    ensureObservers();
    for (Observer observer : observers) {
        observer.update(this);
    }
}

public void publish() {
    notifyObservers();
}
```

7. OOP Principles and Design Patterns

Principle / Pattern	Where It Appears	Explanation
Encapsulation	Most model classes	Fields are private, while getters, setters, and defensive copies protect internal state.
Inheritance	`User`, `Employee`, role classes	Common user and employee behavior is reused by concrete roles.
Polymorphism	`Researcher`, `Observer`, comparators	Services work through interfaces and comparator contracts instead of specific implementations.
Abstraction	`User`, `Employee`, interfaces	Abstract classes and interfaces define shared behavior while hiding concrete details.
Singleton	`Database`	Only one repository instance manages serialized application state.
Observer	`News`, `Observer`, `User`	News items notify subscribed users through the observer interface.
Factory	`UserFactory`	Creates concrete user objects from a type and attributes map.
Strategy	Comparator classes	Different sorting strategies are implemented as separate comparator classes.

8. Project Management Notes

Project responsibilities changed several times because two times some members had retakes, so the team had to redistribute work and adjust deadlines. The final project was completed through a flexible division of responsibilities and repeated review of the submitted code.

8.1 Team Members and Roles

- **Danil:** handled the whole project management process, implemented about 80% of the project code, edited the final structure and diagrams, and checked the work completed by Darkhan.
- **Darkhan:** completed part A and part B, and wrote several classes and features, including `AuthenticationService` and the help feature.

8.2 Task Distribution

At the beginning, the team shared an Excel file and instructions for UML and implementation tasks. Because of retakes we've lost 2 our teammates and responsibilities changed several times. The team adapted by redistributing work around the class diagram, use-case diagram, service classes, console workflow, and final report.

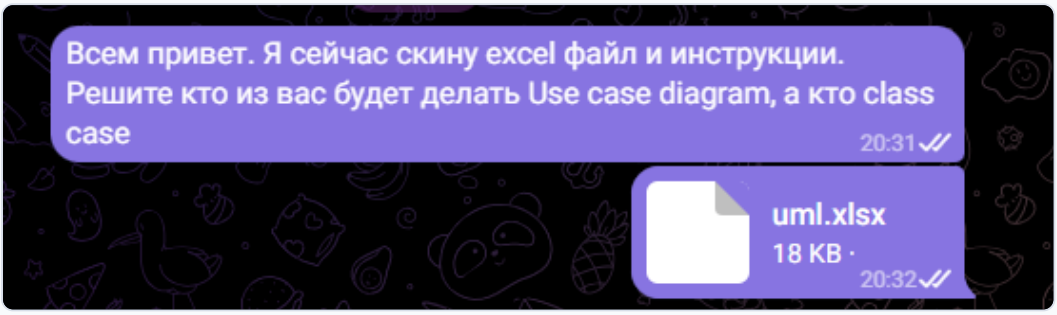
8.3 Risks, Issues, and Resolutions

The main risk was changing availability of team members due to retakes, which affected the planned division of UML and implementation work. As a resolution, Danil had to participate more actively in UML editing and final checking, while Darkhan had to handle all UML diagrams for parts A and B and keep them aligned with the implemented classes.

9. Screenshots

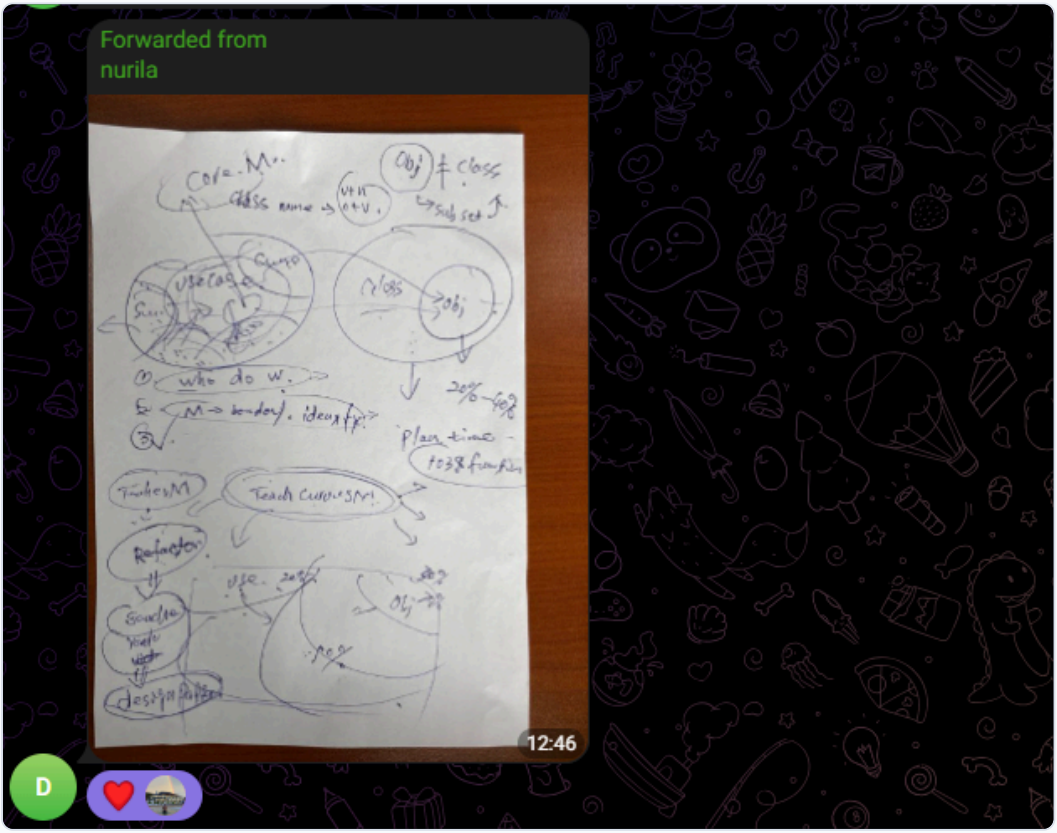
Screenshots and visual notes from team coordination, defense planning, and console workflow.

9.1 Task Distribution Discussion



First coordination screenshot: team members shared the UML instruction file and discussed who would work on the use-case and class diagrams.

9.2 First Defense Insights and Decision Change



Second screenshot: notes from the first defense, where the team reviewed the object model and changed several design decisions.

9.3 Program Workflow

```
PS C:\Users\danil\CODE\Java\project> java -cp out main.Main
University console
Type: help
Demo logins: login A001 admin_hash | login M001 manager_hash | login T001 prof_hash | login S001 stud_hash
> login T001 prof_hash
Nora Professor logged in.
Current user: Nora Professor
> help
Available commands for Teacher:
help
whoami | logout | save | exit
courses
papers date|citations|pages
top | top-year <year> | top-school-year <SCHOOL> <year>
project <topic>
join <projectNumber> <userId>

students
mark <studentId> <courseCode> <att1> <att2> <final>
transcript <studentId>
supervisor <studentId> <teacherId>
request <text>

>
```

Third screenshot: console workflow after logging in as a teacher and opening the help menu, showing available teacher commands for courses, research papers, students, transcripts, supervisors, and requests.

10. Conclusion

The University Management System demonstrates the practical use of object-oriented programming in Java. It separates domain models, services, repositories, enums, exceptions, and comparators into a maintainable package structure. The project applies inheritance for user roles, interfaces for research and notification behavior, custom exceptions for domain constraints, and a singleton repository for persistence.

The system can be extended with a graphical interface, database-backed persistence, stronger password hashing, role-based authorization, automated tests, and additional reporting features. The current implementation already provides a clear foundation for academic workflows, staff operations, research management, and system documentation.

End of OOP Project Report